The Story of Big O

You know how it ends (n^2). Here is how it begins.

Computer Science II • Dr. Jeremy P. Evert

Week 11 optional micro-lab

The story arc

1. Naive code $\rightarrow$ highlight what drives it
2. Count every operation $\rightarrow$ graph it
3. Make n bigger $\rightarrow$ try another CPU
4. A better algorithm $\rightarrow$ a new curve

Evidence: counters and wall-clock time in a CSV you can regenerate on your own machine.

1. The simple bubble sort

```
def bubble_sort(data):
    items = list(data)
    n = len(items)
    for end in range(n - 1, 0, -1):
        swapped = False
        for i in range(end):
            if items[i] > items[i + 1]:
                items[i], items[i + 1] = items[i + 1], items[i]
                swapped = True
    return items
```

Everything Big O will say is already in these ten lines. Where does the n^2 come from?

2. What drives the behavior

```
def bubble_sort(data):
    items = list(data)
    n = len(items)
    for end in range(n - 1, 0, -1):          <- outer loop: n-1 passes
        swapped = False
        for i in range(end):                 <- inner loop: shrinks each pass
            if items[i] > items[i + 1]:      <- compare
                items[i], items[i + 1] = items[i + 1], items[i]   <- swap
                swapped = True
    return items
```

Two nested loops, and the compare and swap inside them. Passes: $n - 1$. Inner trips: $(n-1) + (n-2) + \cdots + 1 = n(n-1)/2$.

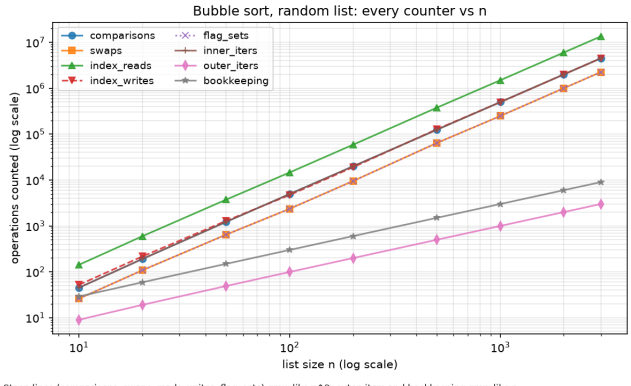
3. What Big O says to ignore

```
def bubble_sort(data):
    items = list(data)                                  <- copy: n steps
    n = len(items)                                      <- len(): 1 step
    for end in range(n - 1, 0, -1):                     <- outer loop: n-1 passes
        swapped = False                                 <- reset flag: n-1 steps
        for i in range(end):                            <- inner loop: shrinks each pass
            if items[i] > items[i + 1]:                 <- compare
                items[i], items[i + 1] = items[i + 1], items[i]   <- swap
                swapped = True                          <- flag set: a constant per swap
    return items                                        <- return: 1 step
```

orange = drives the behavior (n^2) grey = Big O says ignore it

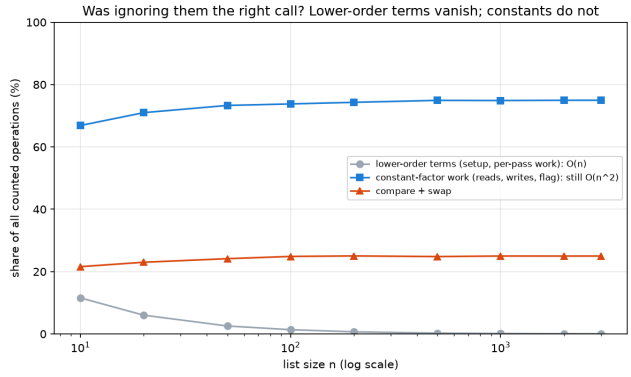
Setup, per-pass bookkeeping, a flag per swap. A claim, not a fact: now we *count* to check it.

4. Count every operation



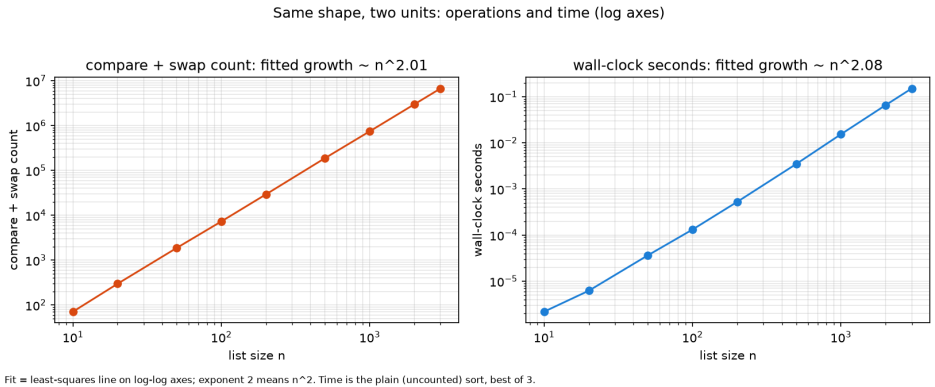
Instrumented sort, one counter per operation type. Compare and swap grow like n^2; passes and setup grow like n.

5. Was ignoring them the right call?



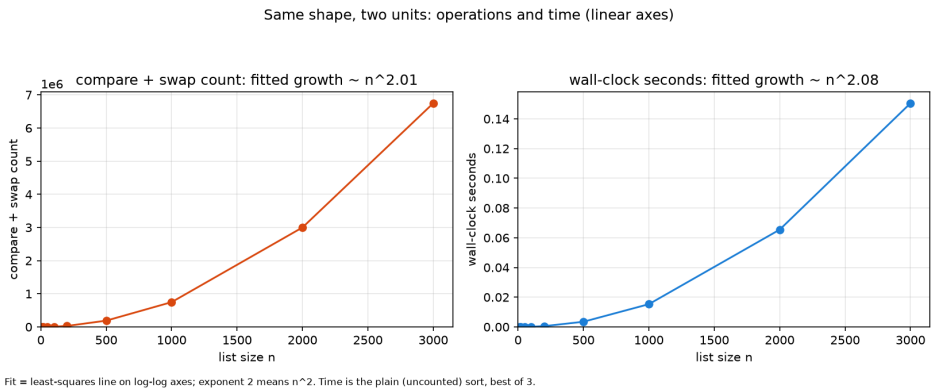
Lower-order terms fade toward 0%. Constant-factor work (reads, writes, flag) stays a fixed share: it changes the speed, not the shape.

6. Operations and time: same shape (log axes)



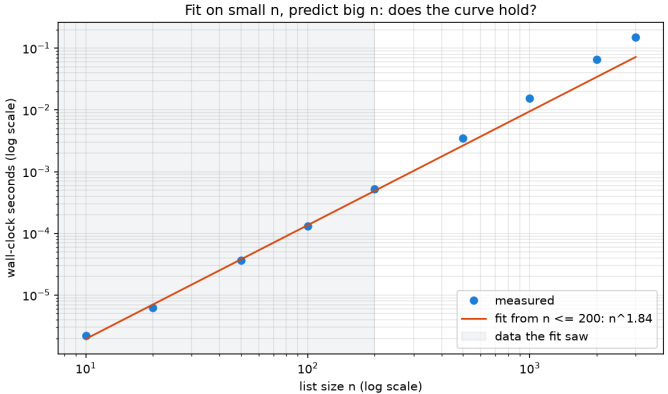
Fitted exponents near 2 for both. The count is exact: compare $= n(n-1)/2$.

6b. The same data on linear axes



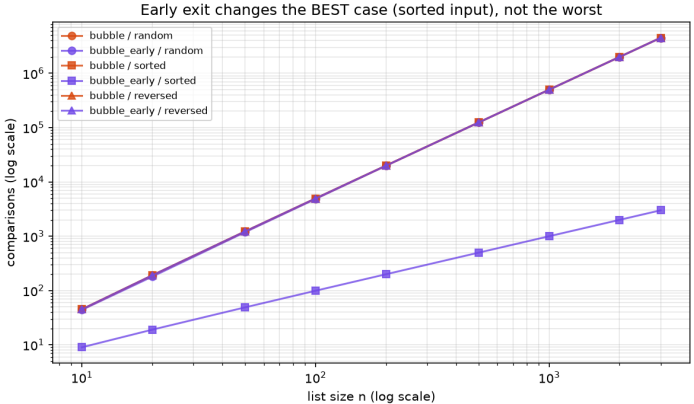
Linear axes show the parabola; log axes show the exponent. Each answers a different question.

7. Fit small, predict big



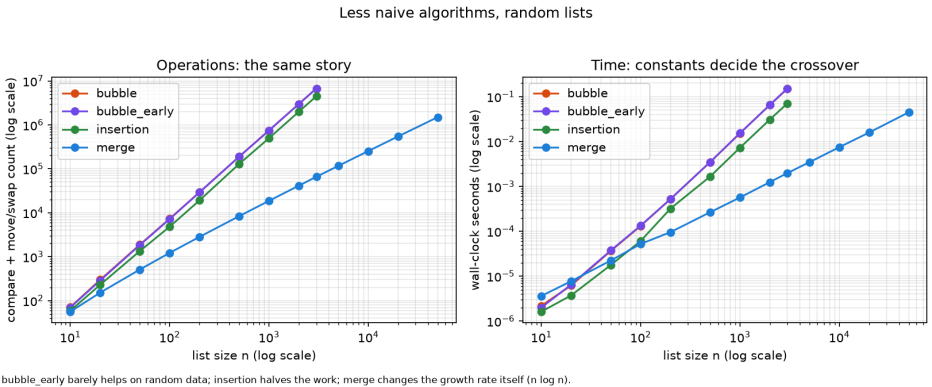
Fit made only from $n \leq 200$, checked against much larger n. Prediction is the test of the model.

8. Less naive: early exit



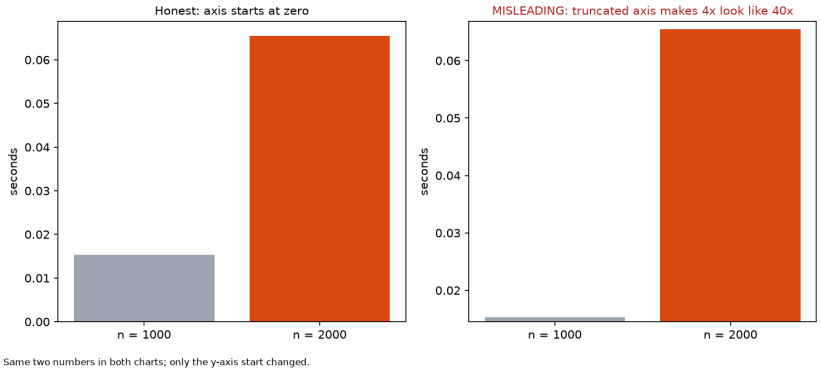
Stopping after a swap-free pass helps the *best* case (sorted input). Random and reversed input are still n^2.

9. A new curve: insertion and merge sort



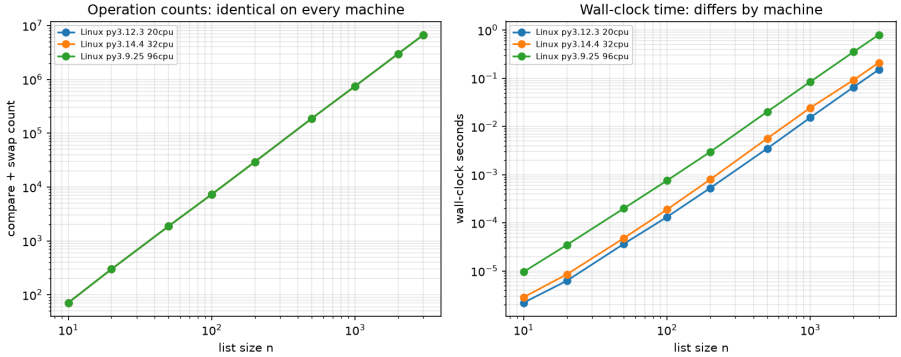
Insertion sort: same growth, about half the work. Merge sort changes the growth itself, $n \log n$. Look for the crossover in time.

Pitfall: the misleading alternative



Same two numbers. A truncated y-axis makes a 4x change look like 40x. Always show the zero baseline for bar charts.

8. Another CPU: counts identical, time not



Same code, three Linux machines (Python 3.12 / 3.14 / 3.9; 20 / 32 / 96 CPUs). All 120 count rows match exactly. Bubble at $n = 3000$: 0.15 s, 0.21 s, 0.80 s. The Python version differs too, so this is not a pure CPU comparison.

Your story, in five parts

Question • Visual • Rationale • Limitation • Evidence

One run, one machine, best-of-N timing, counters add overhead (so time uses the uncounted sort). Claim only what the numbers show.